

Python template:

```
def remove_nth_from_end(head, n):
```

```
    dummy = Node(0)
```

```
    dummy.next = head
```

```
    first = dummy
```

```
    second = dummy
```

```
    # create a gap of n+1
```

```
    for _ in range(n+1):
```

```
        first = first.next
```

```
    # move both pointers  
    while first:
```

```
        first = first.next
```

```
        second = second.next
```

T.C: $O(n)$

S.C: $O(1)$

```
    # remove target
```

```
    second.next = second.next.next
```

```
    return dummy.next
```

Pattern Recognition:

nth node from end, kth node from end, remove
kth from end, find kth from last

For Nth - from - End Rule:

Start

↓

dummy

↓

first & second

↓

Create GAP of n+1

↓

move together

↓

first = None

↓

second = previous of target

↓

Bypass Target

② Palindrome Linked List:

Problem: Given $1 \rightarrow 2 \rightarrow 2 \rightarrow 1 \rightarrow \text{None}$ Determine whether the linked list is a palindrome. a/p: Time

Approach:

Find Middle

↓

Reverse Second half

↓

Compare Halves

Python Template:

```
def is_palindrome(head):
```

```
    if head is None or head.next is None:
        return True
```

```
    # Find middle
```

```
    slow = head
```

```
    fast = head
```

```
    while fast and fast.next:
```

```
        slow = slow.next
```

```
        fast = fast.next.next
```

```
    # Reverse second half
```

```
    prev = None
```

```
    while slow:
```

```
        next_node = slow.next
```

```
        slow.next = prev
```

```
        prev = slow
```

```
        slow = next_node
```

```
    # Compare both halves
```

```
    left = head
```

```
    right = prev
```

```
    while right:
```

```
        if left.data != right.data:
```

```
            return False
```

left = left.next
right = right.next
return Time

T.C: $O(N)$

S.C: $O(1)$

③ Intersection Two Linked List:

Problem: Find the node at which two singly linked lists intersect.

List A: $1 \rightarrow 2 \rightarrow 8 \rightarrow 10 \rightarrow 12$

List B: $5 \rightarrow 6$

Actual Structure:

List A: $1 \rightarrow 2 \rightarrow 8 \rightarrow 10 \rightarrow 12$

List B: $5 \rightarrow 6 \rightarrow 8 \rightarrow 10 \rightarrow 12$

So Intersection Node is 8

Optimal Two-pointer Trick:

$P_1 \rightarrow$ List A $P_2 \rightarrow$ List B (when a pointer reaches end)

Switch P_1 to other list's head:

P_1 reaches None

↓

$P_1 = \text{head B}$

P_1 reaches None

↓

$P_2 = \text{head A}$

Algorithm:

$P_1 = \text{head A}$

$P_2 = \text{head B}$

while $P_1 \neq P_2$:

if P_1 is None:

$P_1 = \text{head B}$

else:

$P_1 = P_1.\text{next}$

if P_2 is None:

$P_2 = \text{head A}$

else: $P_2 = P_2.\text{next}$

return P_1

If intersection exists:

$P_1 == P_2$ at the common node.

If no intersection:

$P_1 == P_2 == \text{None}$

Python Template:

```
def get_intersection_node(head A, head B):
```

```
    p1 = head A
```

```
    p2 = head B
```

```
    while p1 != p2:
```

```
        if p1 is None:
```

```
            p1 = head B
```

```
        else:
```

```
            p1 = p1.next
```

```
        if p2 is None:
```

```
            p2 = head A
```

```
        else:
```

```
            p2 = p2.next
```

```
    return p1
```

T.C: $O(m+n)$

S.C: $O(1)$

pattern:

Two linked lists

+

Common Node

+

Intersection

Immediately:

Two pointers

↓

Head A

Head B

↓

Switch Heads at End

↓

Meet

~ Linked List Completed ~